
REQUIREMENTS NOT MET

N/A

PROBLEMS ENCOUNTERED

N/A

HOMEWORK EXERCISES

- i. **How many TC0 channels are necessary to control all three of the LEDs within the on-board RGB LED package?**

The TC modules are 16-bit counters, which means that one counter can count from \$0000 to \$FFFF. For our purposes however, we can simply have it count up from 0 to \$FF.

We can then compare brightness values of our individual LEDs to the current count, and if our LEDs brightnesses are equal to or higher than the current count, they are set to the ON position, otherwise they are set to the OFF position.

This would make us use one timer (TC0), three compare channels within TC0 (as it has 4 available) with brightness settings for each LED, and we would compare each brightness channel to the timer.

- ii. **In the context of the program specified above, would any difference (theoretically) result from setting the RGB period to be \$FFFF, instead of \$FF?**

Yes, it would let us have a higher level of control over the control, $0xFFFF = 65535$, so we would have 65536 effective brightness levels from 0 to VCC; since we can only represent up to 255, we would be stuck between 0% and $255/65535 = 0.00389164441\%$ brightness. This would give us much less control, but if we had a larger input, potentially better color control, similar to a higher CRI.

PSEUDOCODE/FLOWCHARTS

SECTION 1: Provided Code

Include def file

MAIN:

Set PORTD remapping for timer outputs (upper byte)

Initialize stack pointer

Configure PORTA as inputs (DIP switches)

Configure PORTC as outputs

Configure PD4, PD5, PD6 as outputs ; (RGB LED pins)

Configure Timer D0:

Period = 255

Clock source = DIV1

Single-slope PWM mode

Enable channels A, B, and C

Initialize PWM duty cycles:

Red = 0%

Green = 0%

Blue = 0%

Invert PD4, PD5, PD6 outputs ; (because RGB LED is active-low)

LOOP:

Read DIP switch value from PORTA

Display switch value on PORTC

If Red Select button (PF2) is pressed then

Call RED_SELECT

If Blue Select button (PF3) is pressed then

Call BLUE_SELECT

If Green Select button (PE0) is pressed then

Call GREEN_SELECT

rjmp LOOP

DONE:

Rjmp DONE

; SUBROUTINES

RED_SELECT:

Push registers

Read 8-bit value from PORTA

Store value into Timer Compare A register

Clear high byte of Compare A register

Pop registers

Return

GREEN_SELECT:

Push registers

Read 8-bit value from PORTA

Store value into Timer Compare B register

Clear high byte of Compare B register

Pop registers (reverse)

Return

BLUE_SELECT:

Push registers

Read 8-bit value from PORTA

Store value into Timer Compare C register

Clear high byte of Compare C register

Pop registers (reverse)

Return

; PWM OPERATION

; Timer continuously counts from 0 to 255

; Red LED brightness is determined by Compare A value

; Green LED brightness is determined by Compare B value

; Blue LED brightness is determined by Compare C value

; Lower compare value:

; LED stays on longer

; LED appears brighter

; Smaller compare value:

; LED stays on for less time

; LED appears dimmer

PROGRAM CODE

SECTION 1: PWM Color Control

```
;
; hw2.asm
;
; Created: 6/2/2026 12:15:02 PM
; Author : Evan Baesler
;

/* ABSTRACT: In this program, we are using PWM to control an RGB LED package.
    The program will allow us to alternate between selecting R/G/B
    LED duty cycles, which will be an 8-bit input specified by the
    OOTB SLB input switches. This will let us select a value between
    $00 - $FF for each LED, giving us a standard 0-255 RGB colorscheme
    (24-bit color).

    We will enter the duty-cycle selection for RED using S1 on the SLB
    (PF2), BLUE using S2 on the SLB (PF3), GREEN using S1 on the OOTB MB
    (PE0)

    This duty cycle will then be compared to a timer from 0-255, where
    the respective LED is activated when it's specified duty cycle is
    greater than or equal to the count of the timer.

OBJECTIVES:  (1) Set up Stack Pointer
              (2) Initialize 0% duty cycles for all LEDs
              (3) Set up inputs from PORTA switches
              (4) Set up active-low outputs PD4 (RED), PD5 (GREEN), PD6 (BLUE)
              (5) Set up a DIV1 0-255 counter (faster counting, smoother dimming)
                  utilizing TC0
              (6) Initialize comparison registers
              (7) Create duty cycle selection subroutines utilizing tactile switches (S1's/S2)

*/

.equ TIMERTICKS = 255

.include "ATxmega128A1Udef.inc"

.org 0
rjmp MAIN

.org 0x0100
MAIN:

ldi r16, (0xF)
sts PORTD_REMAP, r16

ldi r16, low(RAMEND)
sts CPU_SPL, r16
ldi r16, high(RAMEND)
sts CPU_SPH, r16

ldi r16, 0
sts PORTA_DIR, r16 ; Sets all of PA to inputs

ldi r16, 0xFF
sts PORTC_DIR, r16
```

```
ldi r16, 0b01110000 ; PD4-6
sts PORTD_DIR, r16 ; Sets respective pins of PD to outputs

ldi r16, low(TIMERTICKS)
sts TCD0_PER, r16 ; 0xFF only takes one byte, do not need PERH
ldi r16, high(TIMERTICKS)
sts TCD0_PER+1, r16

; Initializes the comparison registers
ldi r16, 0
sts TCD0_CCA, r16
sts TCD0_CCA+1, r16
sts TCD0_CCB, r16
sts TCD0_CCB+1, r16
sts TCD0_CCC, r16
sts TCD0_CCC+1, r16

ldi r16, PORT_INVEN_bm ; Inverting bitmask
sts PORTD_PIN4CTRL, r16 ; Flips control (on when duty cycle >= clock)
sts PORTD_PIN5CTRL, r16
sts PORTD_PIN6CTRL, r16

; Sets PWM wavegen mode, and enables three comparison registers for us
ldi r16, (TC_WGMODE_SS_gc | TC0_CCAEN_bm | TC0_CCBEN_bm | TC0_CCCEN_bm)
sts TCD0_CTRLB, r16 ; Store our directions to CTRLB

ldi r16, TC_CLKSEL_DIV1_gc ; faster updates = smoother dimming
sts TCD0_CTRLA, r16

LOOP:

lds r16, PORTA_IN
sts PORTC_OUT, r16

; We will check to enter our selection modes, as our switches are active-low
; we skip whenever the bit for the respective switches are set.

; Check to enter Red PWM Selection (PF2)
lds r16, PORTF_IN
sbrs r16, 2
rcall RED_SELECT

; Check to enter Blue PWM Selection (PF3)
lds r16, PORTF_IN
sbrs r16, 3
rcall BLUE_SELECT

; Check to enter Green PWM Selection (PE0)
lds r16, PORTE_IN
sbrs r16, 0
rcall GREEN_SELECT

; Else LOOP
rjmp LOOP

DONE:
rjmp DONE

; SUBROUTINES:

RED_SELECT:
```

```
push r16 ; store r16's initial value
push r17 ; store r17's initial value

lds r16, PORTA_IN      ; get DIP switches inputs
sts TCD0_CCA, r16      ; update low byte's new duty cycle
ldi r17, 0
sts TCD0_CCA+1, r17    ; clear high byte to update register

pop r17 ; release r17's initial value
pop r16 ; release r16's initial value

ret
```

GREEN_SELECT:

```
push r16 ; store r16's initial value
push r17 ; store r17's initial value

lds r16, PORTA_IN      ; get DIP switches inputs
sts TCD0_CCB, r16      ; update low byte's new duty cycle
ldi r17, 0
sts TCD0_CCB+1, r17    ; clear high byte to update register

pop r17 ; release r17's initial value
pop r16 ; release r16's initial value

ret
```

BLUE_SELECT:

```
push r16 ; store r16's initial value
push r17 ; store r17's initial value

lds r16, PORTA_IN      ; get DIP switches inputs
sts TCD0_CCC, r16      ; update low byte's new duty cycle
ldi r17, 0
sts TCD0_CCC+1, r17    ; clear high byte to update register

pop r17 ; release r17's initial value
pop r16 ; release r16's initial value

ret
```

APPENDIX

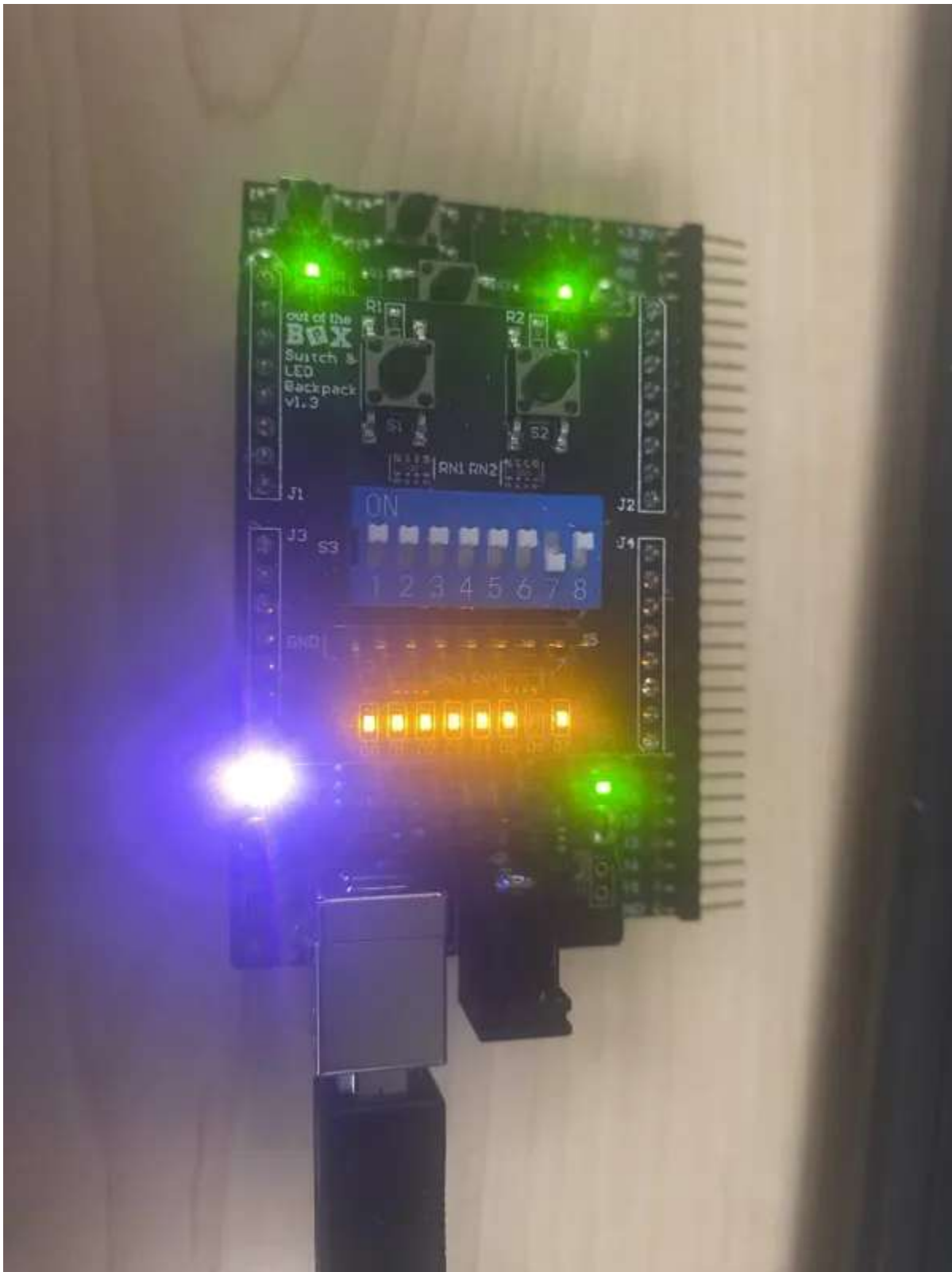


Figure 1: LED with varying Duty Cycle (DC): R 50%, G 25%, B 25%

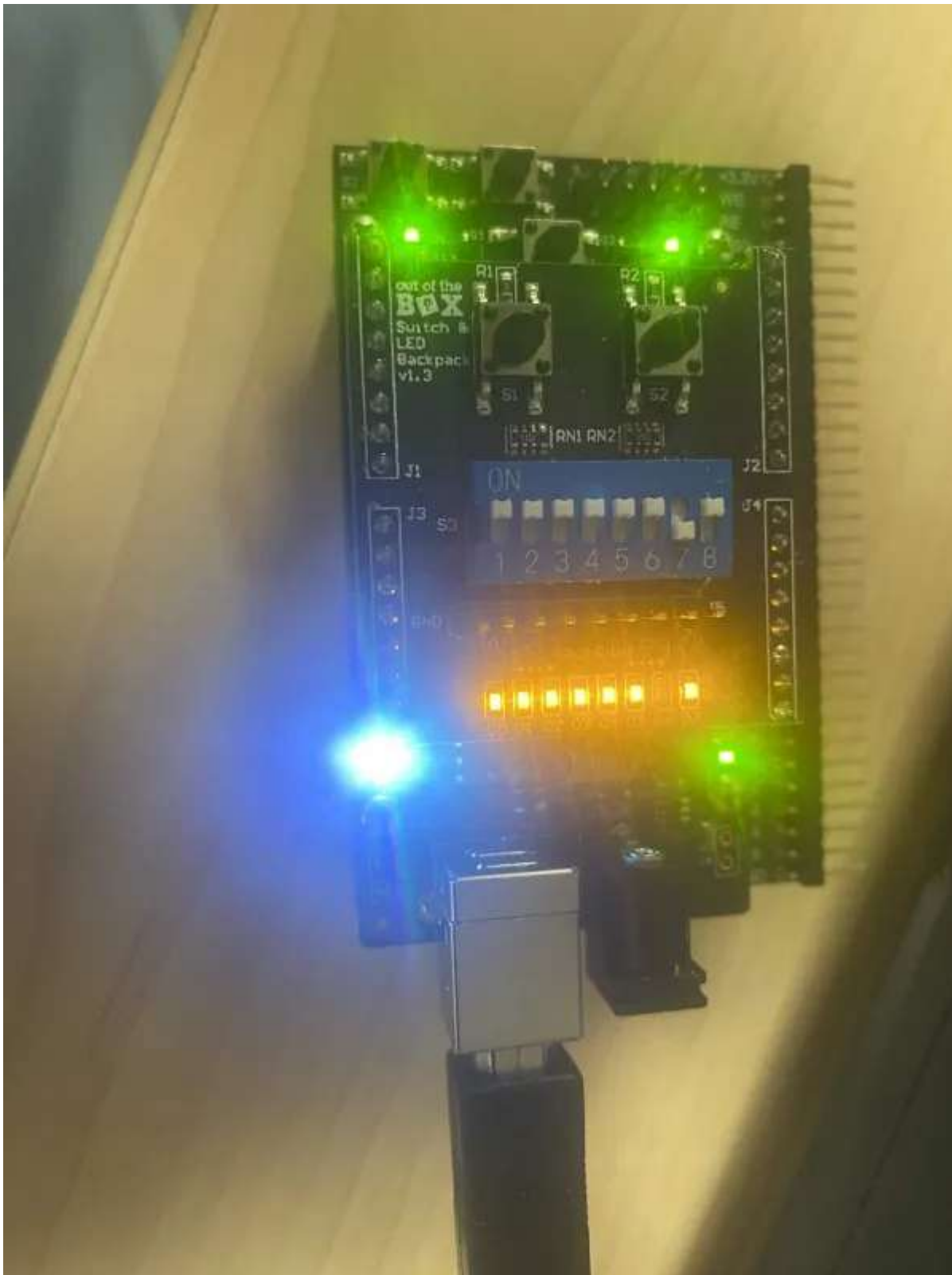


Figure 2: LED with varying Duty Cycle (DC): R 25%, G 50%, B 25%

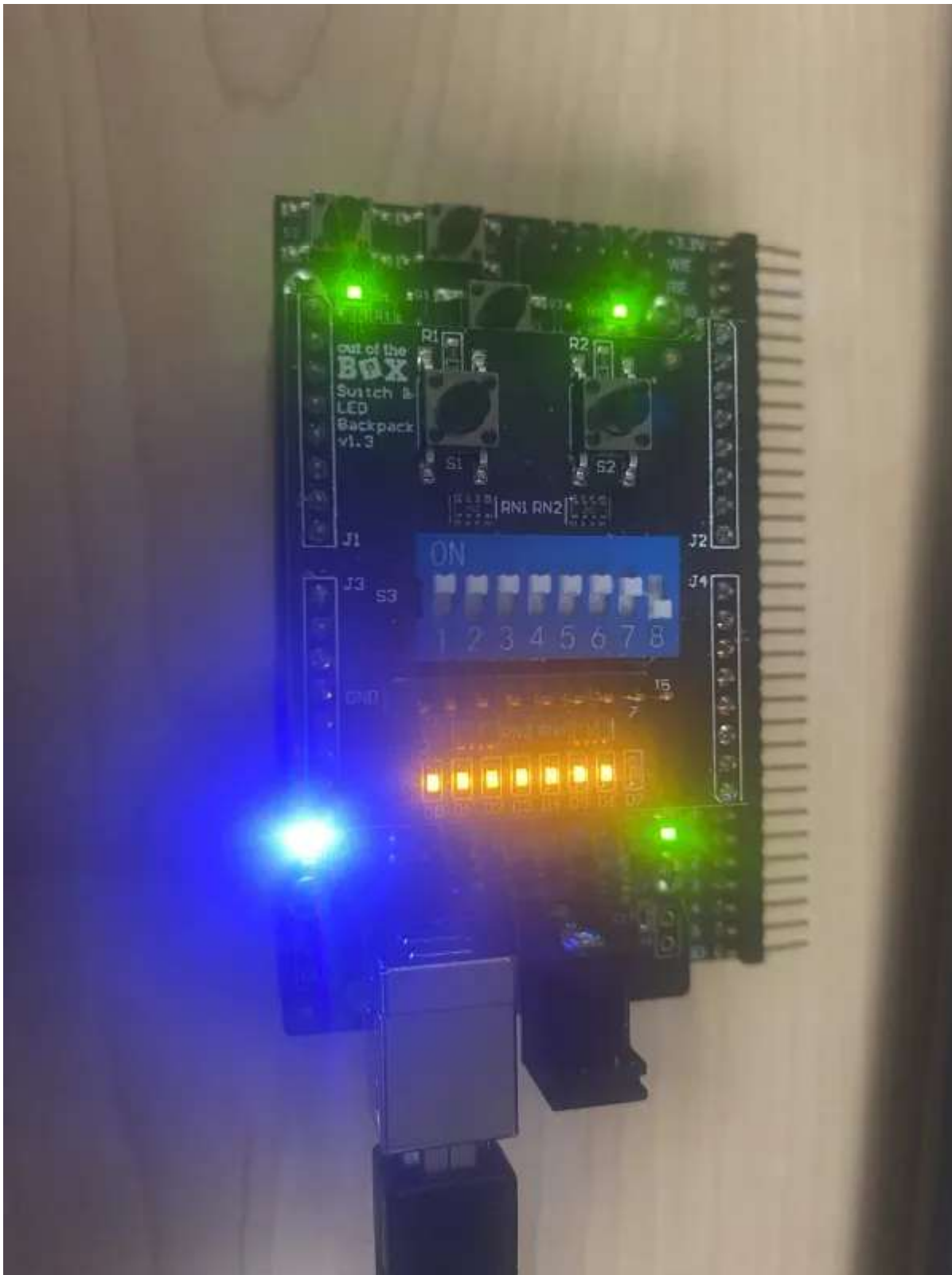


Figure 3: LED with varying Duty Cycle (DC): R 25%, G 25%, B 50%